SkySwallow

开封首-2026-10-20-源代码安全检测报告

开封首-2026-10-20

北京启明星辰信息安全技术有限公司

2026-11-11

1 报告汇总信息

1.1 项目摘要

本次源代码安全测试时间段：

测试工作时间段					
开始时间	2026-11-11 15:17:37	扫描用时	4:38	来源	本地上传
地址	-				

本次源代码安全测试实施人员：

本次源代码安全测试实施人员			
账号	4s1（4s1）	部门	4s
联系方式	11@123.com		

1.2 测试范围

本次源代码安全测试范围：

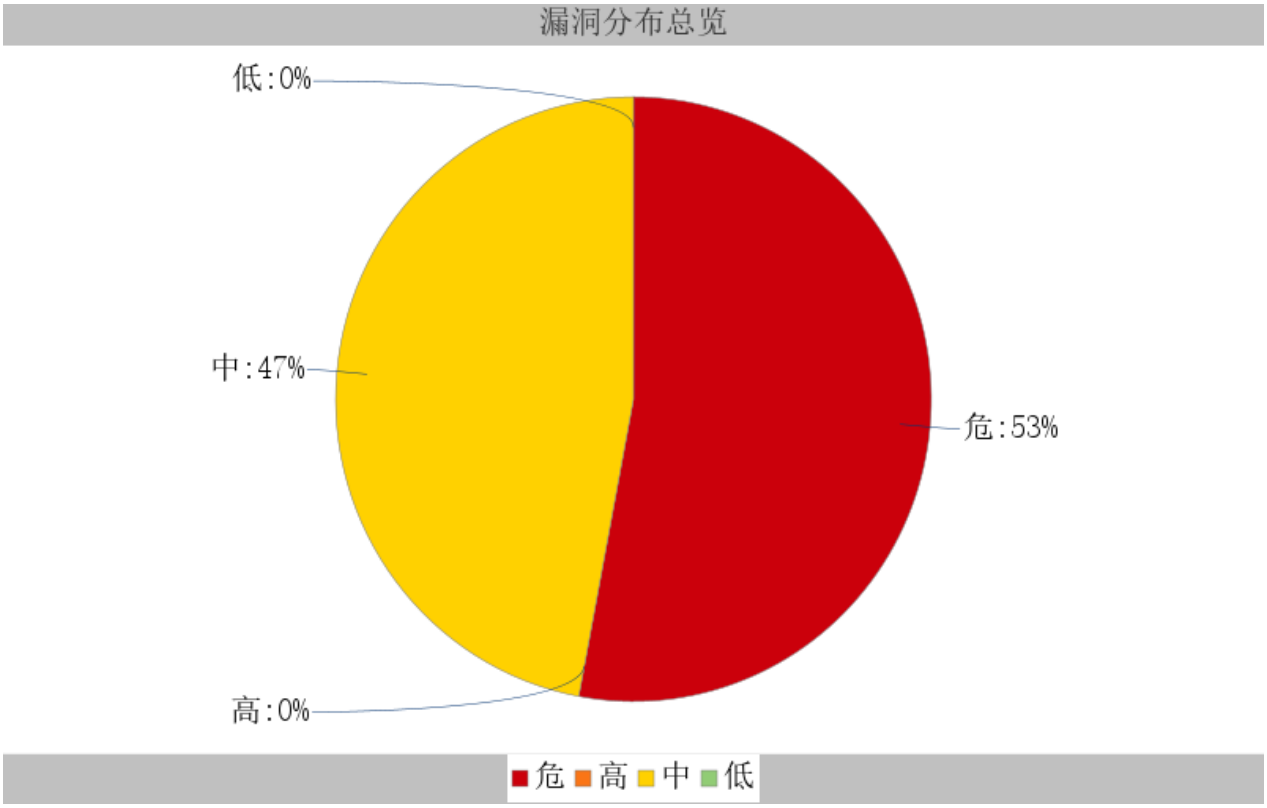
本次源代码安全测试范围			
项目名称	版本	开发语言	扫描文件数(个)
开封首	2026-10-20	GO	615
扫描项目大小(M)	千行漏洞率	漏洞数量	代码行数(行)
40.47	0.25‰	34	133851

1.3 漏洞分布

1.3.1漏洞风险汇总表

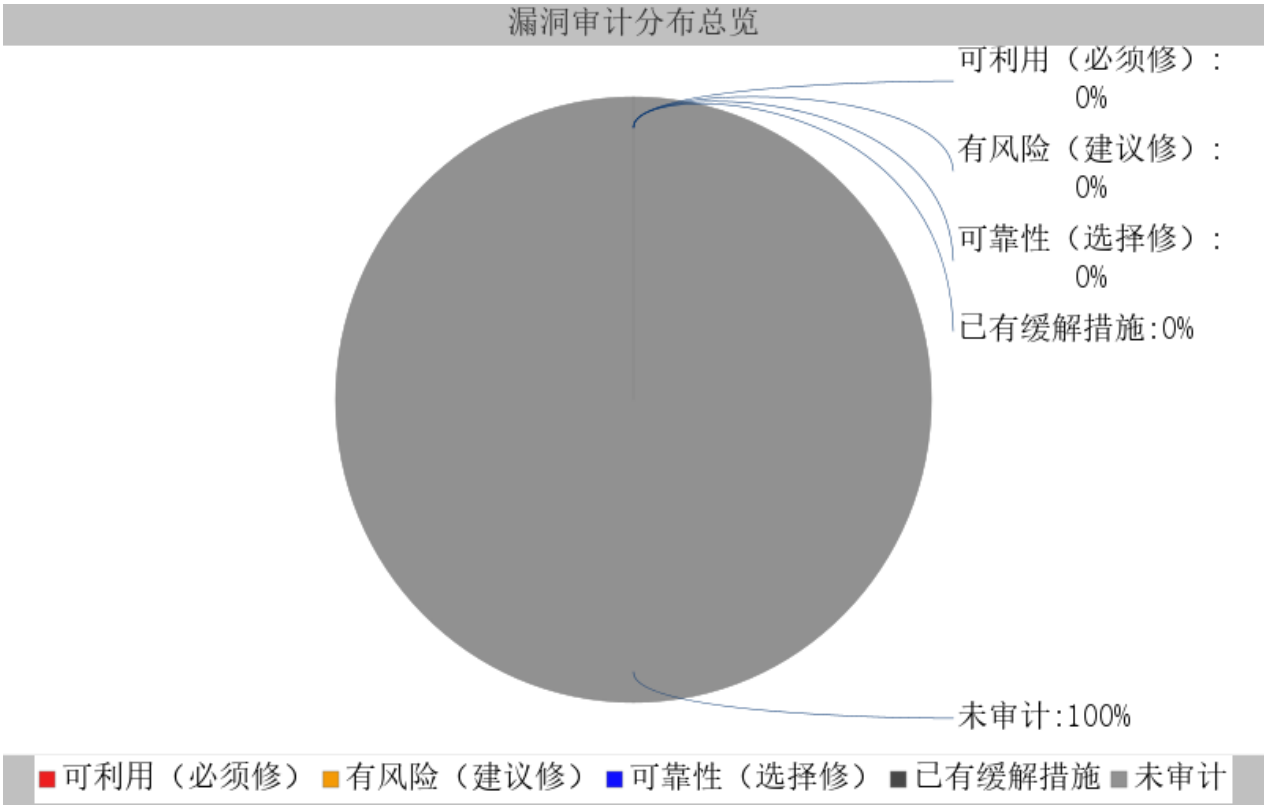
风险类别	漏洞数量
危	18
高	0
中	16
低	0

规范	无



1.3.2漏洞审计汇总

审计状态	漏洞数量
可利用（必须修）	0
有风险（建议修）	0
可靠性（选择修）	0
已有缓解措施	0
未审计	34



2 问题类别汇总清单

漏洞类别		漏洞数量
危	JSON 注入	1
危	密钥管理：硬编码加密密钥	17
中	不当的加密算法：未使用国家商用密码	8
中	私密违反：硬编码URL	6
中	系统信息泄漏：外部	2

3 检测问题明细

风险等级：危 **漏洞数量：18**

漏洞： JSON 注入（1）

解释说明

该方法会将未经验证的输入写入 JSON。攻击者可以将任意元素或属性注入 JSON 实体。

JSON injection 会在以下情况中出现：

1.数据从一个不可信数据源进入程序。

2.将数据写入到 JSON 流。

应用程序通常使用 JSON 来存储数据或发送消息。用于存储数据时，JSON 通常会像缓存数据那样处理，而且可能会包含敏感信息。用于发送消息时，JSON 通常与 RESTful 服务一起使用，并且可以传输敏感信息，例如身份验证凭据。

如果应用程序利用未经验证的输入构造 JSON，则攻击者可以更改 JSON 文档和消息的语义。在相对理想的情况下，攻击者可能会插入无关的元素，导致应用程序在解析 JSON 文档或请求时抛出异常。在更为严重的情况下，例如涉及 JSON Injection，攻击者可能会插入无关的元素，从而允许对 JSON 文档或请求中对业务非常关键的值执行可预见操作。有时，JSON Injection 可以导致 Cross–Site Scripting 或 Dynamic Code Evaluation。

示例 1：以下代码将非特权用户（这些用户具有“默认”角色，与之相反，特权用户具有“管理员”角色）的用户帐户身份验证信息从用户控制的输入变量 username 和 password 序列化为位于 ~/user_info.json 的 JSON 文件：

```
...
func someHandler(w http.ResponseWriter, r *http.Request){
    r.parseForm()
    username := r.FormValue("username")
    password := r.FormValue("password")
    ...
    jsonString := `{
    "username":"` + username + `",
    "role":"default"
    "password":"` + password + `",
    }`
    ...
    f, err := os.Create("~/user_info.json")
    defer f.Close()

    jsonEncoder := json.NewEncoder(f)
    jsonEncoder.Encode(jsonString)
}
```

由于代码使用字符串串联来执行 JSON 序列化，将不会对 username 和 password 中的不可信赖数据进行验证以转义与 JSON 相关的特殊字符。这样，用户就可以任意插入 JSON 密钥，这可能会更改已序列化的 JSON 结构。在本例中，如果非特权用户 mallory（密码为 Evil123!）在输入其用户名时附加了 ","role":"admin，则最终保存到 ~/user_info.json 的 JSON 将为：

```
{
"username":"mallory",
"role":"default",
"password":"Evil123!",
"role":"admin"
}
```

在没有进一步验证反序列化 JSON 值是否有效的情况下，应用程序会无意中为用户分配 mallory“管理员”特权。

修复建议

在将用户提供的数据写入 JSON 时，请遵循以下准则：

1.不要使用从用户输入派生的名称创建 JSON 属性。

2.确保使用安全的序列化函数（能够以单引号或双引号分隔不可信赖的数据，并且避免任何特殊字符）执行对 JSON 的所有序列化操作。

防止 JSON 注入的最佳方法是采用一些间接手段：创建一份必须由用户选择的合法资源名称的列表。通过这种方法，就不能直接使用用户输入来指定资源名称。

有时，这种方法并不可行，因为这样一份合法资源名称的列表太大，管理难度大。因此，在这种情况下，程序员通常会采用黑名单的办法。在输入之前，黑名单会有选择地拒绝或避免潜在的危险字符。但是，任何这样一份黑名单都不可能是完整的，而且将随着时间的推移而过时。更好的方法是创建一份白名单，允许其中的字符出现在资源名称中，且只接受完全由这些被认可的字符组成的输入。

```
import "encoding/json"

type UserInfo struct {
    Username string `json:"username"`
    Role     string `json:"role"`
    Password string `json:"password"` // 仅作演示，生产环境不应明文存密码
}

func someHandler(w http.ResponseWriter, r *http.Request) {
    r.ParseForm()
    username := r.FormValue("username")
    password := r.FormValue("password")

    // 安全：使用结构体
    user := UserInfo{
        Username: username,
        Role:     "default", // 安全：角色由服务端固定
        Password: password,
    }

    // 序列化：库会自动处理特殊字符转义
    jsonData, err := json.Marshal(user)
    if err != nil {
        http.Error(w, err.Error(), 500)
        return
    }

    // 写入文件
    err = os.WriteFile("user_info.json", jsonData, 0644)
```

```
    if err != nil {
        http.Error(w, err.Error(), 500)
        return
    }
}
```

漏洞明细

编号： 开封首 – 2026–10–20 – 1/1 （危 – JSON 注入）

源码定位： backend/handler/share/tgt_login.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/handler/share/tgt_login.go：222 –-> Read resp.Body

backend/handler/share/tgt_login.go：222 –-> LimitReader(0 : return)

backend/handler/share/tgt_login.go：222 –-> ReadAll(0 : return.value_0)

backend/handler/share/tgt_login.go：222 –-> Assignment to raw

backend/handler/share/tgt_login.go：229 –-> Unmarshal(0[] : 1)

backend/handler/share/tgt_login.go：131 –-> post(4)

backend/handler/share/tgt_login.go：140 –-> TrimSpace(0 : return)

backend/handler/share/tgt_login.go：140 –-> Assignment to loginName

backend/handler/share/tgt_login.go：154 –-> Assignment to LoginName

backend/handler/share/tgt_login.go：153 –-> Assignment to ~return~variable.value_0

backend/handler/share/tgt_login.go：153 –-> Return ~return~variable

backend/handler/share/tgt_login.go：92 –-> loginVerify(return.value_0)

backend/handler/share/tgt_login.go：92 –-> Assignment to sessionUser

backend/handler/share/tgt_login.go：97 –-> hasKfsUserPermission(2.LoginName)

backend/handler/share/tgt_login.go：186 –-> ~golang~map~put(2 : 0['loginName'])

backend/handler/share/tgt_login.go：184 –-> ~golang~map~put(2 : 0 ['generalArgument'])

backend/handler/share/tgt_login.go：183 –-> post(3)

backend/handler/share/tgt_login.go：208 –-> Marshal(0)

攻击源：

```
219         return err
220     }
221     defer resp.Body.Close()
222     raw, err := io.ReadAll(io.LimitReader(resp.Body, 1<<20))
223     if err != nil {
224         return err
225     }
```

爆发点：

```
205}
206
207func (c *cloudAuthClient) post(ctx context.Context, path string, body any, out any) error {
208   payload, err := json.Marshal(body)
209   if err != nil {
210       return err
211   }
```

漏洞： 密钥管理：硬编码加密密钥 （17）

解释说明

硬编码加密密钥可能会削弱安全性，一旦出现安全问题将无法轻易修正。
请勿对加密密钥进行硬编码，因为这样所有项目开发人员都能查看该加密密钥，而且还会大大增加解决问题的难度。一旦代码投入使用，要更改加密密钥，就必须进行软件修补。如果受加密密钥保护的帐户被盗用，系统所有者将必须在安全性和可用性之间做出选择。

示例 1：以下代码使用硬编码加密密钥：

```
...
key := []byte("lakdsljkalkjlksd");
block, err := aes.NewCipher(key)
...
```

任何可访问该代码的人都能访问加密密钥。应用程序一经发布，除非对程序进行修补，否则将无法更改加密密钥。雇员可以利用手中掌握的信息访问权限入侵系统。如果攻击者可以访问应用程序的可执行文件，他们就可以提取加密密钥值。

修复建议

加密密钥不应为空。加密密钥应加以模糊化，并在外部数据源中进行管理。在系统中的任何位置采用明文的形式存储加密密钥，会导致任何有足够权限的人均可读取并无意中误用加密密钥。
请勿对加密密钥进行硬编码，而应对加密密钥加以模糊化，并在外部资源文件中进行管理。如果在系统中采用明文的形式存储加密密钥，任何有足够权限的人即可读取加密密钥，还可能误用这些密码。
借助专门的密钥管理系统来生成、存储和分发加密密钥。像 AWS KMS、Google Cloud KMS 这类云服务提供商提供的 KMS，能帮助安全地管理密钥。
把加密密钥存储在环境变量中，在代码里通过读取环境变量来获取密钥。
将加密密钥存于配置文件中，并且对配置文件进行严格的权限管理，确保只有授权人员能够访问。

上述案例修复之后的代码如下：

```
// 从环境变量中读取加密密钥
encryptionKey := os.Getenv("ENCRYPTION_KEY")

if encryptionKey == "" {
    fmt.Println("Encryption key not set in environment variable ENCRYPTION_KEY")
    return
}

// AES-256需要32字节的密钥长度
const requiredKeyLength = 32
if len(encryptionKey) != requiredKeyLength {
    fmt.Printf("Invalid encryption key length. Expected %d bytes, got %
d\n", requiredKeyLength, len(encryptionKey))
    return
}

// 创建一个新的AES cipher块
block, err := aes.NewCipher([]byte(encryptionKey))
if err != nil {
    fmt.Println("Error creating cipher:", err)
    return
}

fmt.Println("Cipher created successfully with dynamic encryption key.")
```

漏洞明细

编号： 开封首 – 2026-10-20 – 1/17 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki-src-5f7df49a/web/packages/ui/src/chatTopNStorage.ts

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki-src-5f7df49a/web/packages/ui/src/chatTopNStorage.ts：1

代码片段：

```
1  export const CHAT_TOP_N_STORAGE_KEY = 'panda_wiki_chat_top_n';
2
3  export const DEFAULT_CHAT_TOP_N = 10;
4
```

编号： 开封首 - 2026-10-20 - 2/17（危 - 密钥管理：硬编码加密密钥）

源码定位： PandaWiki-src-5f7df49a/docker-compose.yml

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki-src-5f7df49a/docker-compose.yml：182

代码片段：

```
179       - MQ_NATS_URL=nats://panda-wiki-nats:4222
180       - MQ_NATS_USER=panda-wiki
181       - MQ_NATS_PASSWORD=${NATS_PASSWORD}
182       - OSS_MINIO_ACCESS_KEY=s3panda-wiki
183       - OSS_MINIO_SECRET_KEY=${S3_SECRET_KEY}
184       - OSS_MINIO_ENDPOINT=panda-wiki-minio:9000
185     networks:
```

编号： 开封首 - 2026-10-20 - 3/17（危 - 密钥管理：硬编码加密密钥）

源码定位： PandaWiki-src-5f7df49a/backend/store/pg/migration/000032_create_system_settings.up.sql

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki-src-5f7df49a/backend/store/pg/migration/000032_create_system_settings.up.sql：29

代码片段：

```
26      ),
27      'Model setting mode configuration'
28  WHERE NOT EXISTS (
29      SELECT 1 FROM system_settings WHERE key = 'model_setting_mode'
30  );
```

编号： 开封首 – 2026–10–20 – 4/17 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki–src–5f7df49a/web/packages/ui/src/chatQaModeStorage.ts

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki–src–5f7df49a/web/packages/ui/src/chatQaModeStorage.ts：6

代码片段：

```
3   /** 同页内切换模式时通知问答弹窗等订阅方更新样式 */
4   export const QA_APP_MODE_CHANGE_EVENT = 'panda–wiki:qa–app–
mode';
5
6   export const CHAT_QA_MODE_STORAGE_KEY =
'panda_wiki_qa_app_mode';
7
8   export const DEFAULT_QA_APP_MODE: QaAppMode = 'training';
9
```

编号： 开封首 – 2026–10–20 – 5/17 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki–src–5f7df49a/backend/store/pg/migration
/000041_add_standard_terms_system_setting.up.sql

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki–src–5f7df49a/backend/store/pg/migration
/000041_add_standard_terms_system_setting.up.sql：7

代码片段：

```
4       '{"items":[]}'::jsonb,
5       'Global standard terms and aliases'
6   WHERE NOT EXISTS (
7       SELECT 1 FROM system_settings WHERE key = 'standard_terms'
8   );
9
```

编号： 开封首 – 2026–10–20 – 6/17 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki–src–5f7df49a/backend/domain/auth.go

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki–src–5f7df49a/backend/domain/auth.go：17

代码片段：

```
14  )
15
16  const (
17      SessionCacheKey = "_session_store"
18      SessionName     = "_pw_auth_session"
19  )
20
```

编号： 开封首 – 2026–10–20 – 7/17 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki–src–5f7df49a/docker–compose.yml

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki–src–5f7df49a/docker–compose.yml：62

代码片段：

```
59        – MQ_NATS_SERVER=nats://panda–wiki–nats:4222
60        – POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
61        – REDIS_PASSWORD=${REDIS_PASSWORD}
62        – S3_ACCESS_KEY=${S3_ACCESS_KEY:–s3panda–wiki}
63        – S3_SECRET_KEY=${S3_SECRET_KEY}
```

```
64        - S3_ENDPOINT=${S3_ENDPOINT:-panda-wiki-minio:9000}
65        - JWT_SECRET=${JWT_SECRET}
```

编号： 开封首 - 2026-10-20 - 8/17 （危 - 密钥管理：硬编码加密密钥）

源码定位： PandaWiki-src-5f7df49a/backend/consts/system_setting.go

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki-src-5f7df49a/backend/consts/system_setting.go：8

代码片段：

```
5   const (
6       SystemSettingModelMode     SystemSettingKey = "model_setting_mode"
7       SystemSettingUpload        SystemSettingKey = "upload"
8       SystemSettingStandardTerms SystemSettingKey = "standard_terms"
9       SystemSettingTestEnv       SystemSettingKey = "test_env"
10      SystemSettingCloudAuth     SystemSettingKey = "cloud_auth"
11  )
```

编号： 开封首 - 2026-10-20 - 9/17 （危 - 密钥管理：硬编码加密密钥）

源码定位： PandaWiki-src-5f7df49a/backend/consts/system_setting.go

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki-src-5f7df49a/backend/consts/system_setting.go：6

代码片段：

```
3   type SystemSettingKey string
4
5   const (
6       SystemSettingModelMode     SystemSettingKey = "model_setting_mode"
7       SystemSettingUpload        SystemSettingKey = "upload"
8       SystemSettingStandardTerms SystemSettingKey = "standard_terms"
9       SystemSettingTestEnv       SystemSettingKey = "test_env"
```

编号： 开封首 – 2026–10–20 – 10/17 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki–src–5f7df49a/backend/store/pg/migration/000041_add_standard_terms_system_setting.down.sql

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki–src–5f7df49a/backend/store/pg/migration/000041_add_standard_terms_system_setting.down.sql：1

代码片段：

```
1  DELETE FROM system_settings WHERE key = 'standard_terms';
2
```

编号： 开封首 – 2026–10–20 – 11/17 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki–src–5f7df49a/backend/domain/api_token.go

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki–src–5f7df49a/backend/domain/api_token.go：35

代码片段：

```
32  type contextKey string
33
34  const (
35      CtxAuthInfoKey contextKey = "ctx_auth_info"
36  )
37
38  func GetAuthInfoFromCtx(c context.Context) *CtxAuthInfo {
```

编号： 开封首 – 2026–10–20 – 12/17 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki–src–5f7df49a/docker–compose.yml

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki-src-5f7df49a/docker-compose.yml：88

代码片段：

```
85        - MQ_NATS_SERVER=nats://panda-wiki-nats:4222
86        - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
87        - REDIS_PASSWORD=${REDIS_PASSWORD}
88        - S3_ACCESS_KEY=${S3_ACCESS_KEY:-s3panda-wiki}
89        - S3_SECRET_KEY=${S3_SECRET_KEY}
90        - S3_ENDPOINT=${S3_ENDPOINT:-panda-wiki-minio:9000}
91        - JWT_SECRET=${JWT_SECRET}
```

编号： 开封首 - 2026-10-20 - 13/17 （危 - 密钥管理：硬编码加密密钥）

源码定位： PandaWiki-src-5f7df49a/web/app/next.config.ts

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki-src-5f7df49a/web/app/next.config.ts：47

代码片段：

```
44          source: '/cap@0.0.6/:path*',
45          headers: [
46            {
47              key: 'Cache-Control',
48              value: 'public, max-age=31536000, must-revalidate',
49            },
50          ],
```

编号： 开封首 - 2026-10-20 - 14/17 （危 - 密钥管理：硬编码加密密钥）

源码定位： PandaWiki-src-5f7df49a/backend/consts/system_setting.go

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki-src-5f7df49a/backend/consts/system_setting.go：10

代码片段：

```
7       SystemSettingUpload        SystemSettingKey = "upload"
8       SystemSettingStandardTerms SystemSettingKey = "standard_terms"
9       SystemSettingTestEnv       SystemSettingKey = "test_env"
10      SystemSettingCloudAuth     SystemSettingKey = "cloud_auth"
11  )
12
```

编号： 开封首 – 2026–10–20 – 15/17 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki–src–5f7df49a/backend/docker–compose.services.yml

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki–src–5f7df49a/backend/docker–compose.services.yml：73

代码片段：

```
70      restart: unless–stopped
71      command: server /data ––console–address ":9001"
72      environment:
73        MINIO_ROOT_USER: ${S3_ACCESS_KEY:–s3panda–wiki}
74        MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD}
75      ports:
76        – "9000:9000"   # API 端口
```

编号： 开封首 – 2026–10–20 – 16/17 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki–src–5f7df49a/docker–compose.yml

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki–src–5f7df49a/docker–compose.yml：141

代码片段：

```
138     ports:
139       – "9000:9000"
140     environment:
141       – MINIO_ACCESS_KEY=s3panda–wiki
```

```
142         – MINIO_SECRET_KEY=${S3_SECRET_KEY}
143       networks:
144         panda–wiki:
```

编号： 开封首 – 2026–10–20 – 17/17 （危 – 密钥管理：硬编码加密密钥）

源码定位： PandaWiki–src–5f7df49a/backend/domain/license.go

审计状态： 未审计

批注信息： 无

路径追踪：

PandaWiki–src–5f7df49a/backend/domain/license.go：8

代码片段：

```
5       "encoding/json"
6   )
7
8   const ContextKeyEditionLimitation contextKey = "edition_limitation"
9
10  type BaseEditionLimitation struct {
11      MaxKb                int   `json:"max_kb"`                // 知识库站点数量
```

风险等级：中　　漏洞数量：16

漏洞： 不当的加密算法：未使用国家商用密码（8）

解释说明

商用密码是指对不涉及国家秘密内容的信息进行加密保护或者 安全认证 所使用的 密码技术 和密码产品。 商用密码技术 是商用密码的核心，是信息化时代社会团体、组织、企事业单位和个人用于保护自身权益的重要工具。 国家将商用密码技术列入国家秘密，任何单位和个人都有责任和义务保护商用密码技术的秘密。

Incorrect Cryptographic Algorithm 指的是在使用加密算法时，选择了不合适、不安全或者已被破解的加密算法，从而导致数据的机密性、完整性和可用性受到威胁。加密算法是保障数据安全的重要手段，不同的算法在安全性、性能和适用场景上有所不同。如果使用了不安全的算法，攻击者可能会通过分析密文来破解出明文，或者篡改密文而不被察觉。

缺陷示例：

```
...
//使用过时的DES算法
KeyGenerator keyGen = KeyGenerator.getInstance("DES");
```

```
SecretKey secretKey = keyGen.generateKey();

Cipher cipher = Cipher.getInstance("DES");
cipher.init(Cipher.ENCRYPT_MODE, secretKey);

String plaintext = "Hello, World!";
byte[] encryptedBytes = cipher.doFinal(plaintext.getBytes(StandardCharsets.
UTF_8));
String encryptedText = Base64.getEncoder().encodeToString(encryptedBytes);
...
```

在上述代码中，使用了过时的 DES 算法进行加密，由于 DES 算法的密钥长度较短，容易被暴力破解，存在安全风险。

修复建议

所有算法中其中的ZUC、SM1、SM2、SM3、SM4、SM7、SM9是国内认可的商密算法，其余为国际算法。截止至本文当日，SM1和SM7仍然未公开，使用SM1和SM7只能借助于具备商密资质的安全芯片。ZUC、SM2、SM3、SM4、SM9算法已经公开，网上也可找到公开的源码。实际使用中尽量使用已经公开的商密算法，其他未公开的商密算法和国际算法只有在特定需求中才使用。在商密产品认证过程中，原则上都要使用商密算法，尽量避免使用国际算法。

上述案例修复之后的代码如下：

```
                    // 使用安全的 AES 算法
    KeyGenerator keyGen = KeyGenerator.getInstance("AES");
    keyGen.init(256); // 使用 256 位密钥
    SecretKey secretKey = keyGen.generateKey();

    Cipher cipher = Cipher.getInstance("AES");
    cipher.init(Cipher.ENCRYPT_MODE, secretKey);

    String plaintext = "Hello, World!";
    byte[] encryptedBytes = cipher.doFinal(plaintext.getBytes(StandardCharsets.
UTF_8));
    String encryptedText = Base64.getEncoder().encodeToString(encryptedBytes);
```

在修复后的代码中，使用了安全的 AES 算法进行加密，并将密钥长度设置为 256 位。AES 算法具有较高的安全性和性能，能够有效保护数据的机密性。

漏洞明细

编号： 开封首 – 2026–10–20 – 1/8（中 – 不当的加密算法：未使用国家商用密码）

源码定位： sdk/rag/document.go

审计状态： 未审计

批注信息： 无

路径追踪：

sdk/rag/document.go：130 –-> Copy()

代码片段：

```
127     return err
128 }
129 defer out.Close()
130 _, err = io.Copy(out, resp.Body)
131 return err
132}
```

编号： 开封首 – 2026–10–20 – 2/8 （中 – 不当的加密算法：未使用国家商用密码）

源码定位： sdk/rag/document.go

审计状态： 未审计

批注信息： 无

路径追踪：

sdk/rag/document.go：280 –-> Copy()

代码片段：

```
277 if err != nil {
278     return nil, err
279 }
280 if _, err := io.Copy(fw, tmpFile); err != nil {
281     return nil, err
282 }
```

编号： 开封首 – 2026–10–20 – 3/8 （中 – 不当的加密算法：未使用国家商用密码）

源码定位： sdk/rag/document.go

审计状态： 未审计

批注信息： 无

路径追踪：

sdk/rag/document.go：53 –-> Copy()

代码片段：

```
50      if err != nil {
51          return nil, err
52      }
53      if _, err := io.Copy(fw, file); err != nil {
54          return nil, err
55      }
56  }
```

编号： 开封首 – 2026–10–20 – 4/8 （ 中 – 不当的加密算法：未使用国家商用密码 ）

源码定位： backend/telemetry/sm4.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/telemetry/sm4.go：33 –-> Seal()

代码片段：

```
30      return "", err
31  }
32
33  ciphertext := gcm.Seal(nonce, nonce, data, nil)
34  return base64.StdEncoding.EncodeToString(ciphertext), nil
35}
```

编号： 开封首 – 2026–10–20 – 5/8 （ 中 – 不当的加密算法：未使用国家商用密码 ）

源码定位： sdk/rag/document.go

审计状态： 未审计

批注信息： 无

路径追踪：

sdk/rag/document.go：402 –-> Copy()

代码片段：

```
399    if err != nil {
400        return err
401    }
402    if _, err := io.Copy(fw, tmpFile); err != nil {
403        return err
404    }
```

编号： 开封首 – 2026–10–20 – 6/8 （中 – 不当的加密算法：未使用国家商用密码）

源码定位： backend/utils/processor.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/utils/processor.go：66 –-> Copy()

代码片段：

```
63    if node == nil {
64        return nil
65    }
66    if _, err := io.Copy(t.result, node.buf); err != nil {
67        return err
68    }
69    for _, son := range node.son {
```

编号： 开封首 – 2026–10–20 – 7/8 （中 – 不当的加密算法：未使用国家商用密码）

源码定位： backend/utils/epub.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/utils/epub.go：143 –-> Copy()

代码片段：

```
140        e.logger.Debug("add File", "file name", clearFileName(e.resourcesIdMap
[itemRef.IDRef].Href))
141        if r, ok := res[clearFileName(e.resourcesIdMap[itemRef.IDRef].Href)]; ok {
```

```
142         result.WriteString("<span id=" + title + "></span>\n\n")
143         if _, err := io.Copy(result, r); err != nil {
144             return "", nil, err
145         }
146         result.WriteString("\n\n")
```

编号： 开封首 – 2026–10–20 – 8/8 （ 中 – 不当的加密算法：未使用国家商用密码 ）

源码定位： backend/utils/epub.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/utils/epub.go：124 –-> Copy()

代码片段：

```
121   result := bytes.NewBuffer(nil)
122   for _, href := range p.Guide.References {
123       if r, ok := res[clearFileName(href.Href)]; ok {
124           if _, err := io.Copy(result, r); err != nil {
125               return "", nil, err
126           }
127           result.WriteString("\n\n")
```

漏洞： 私密违反：硬编码URL （ 6 ）

解释说明

代码中存在硬编码URL地址，这很有可能是开发人员遗留的测试信息。但如果有真实的URL地址在代码注释中，那么所有开发人员都将有意无意中获取到这些敏感信息，从而产生私密信息泄露的风险。 同时，由于工作流程不合规范或者在程序发布过程中存在疏忽，这些包含了硬编码URL地址的代码一旦发布上线，存在于正式运行的系统中，则很有可能被攻击者所利用。

当参考的 URL 发生变化时，由于注释中的 URL 没有被纳入代码的管理和更新机制，开发者可能会忘记更新注释中的 URL，导致注释中的信息与实际情况不符，给后续的开发者带来困扰。

在某些行业或组织中，对代码中的敏感信息有严格的合规要求。硬编码在注释中的 URL 可能违反这些合规要求，导致潜在的法律风险。

例：以下代码中使用硬编码URL地址

String payServerUrl = "http://www.pay.com"

修复建议

不要在配置文件中配置URL地址，如果是用于测试的代码，在上线前删除掉。

如果 URL 包含敏感信息，不要将其硬编码在注释中。可以使用更安全的方式来存储和管理这些信息，如使用环境变量、配置文件等。

建立代码审查机制，定期审查代码中的注释，确保注释中的 URL 是最新的。如果 URL 发生变化，及时更新注释。

如果无法避免在注释中使用 URL，可以对 URL 进行脱敏处理，例如只保留域名部分，不显示具体的路径和参数。

上述案例修复之后的代码如下：

```
// 参考文档：https://example.com
public class ExampleClass {
    // 类的具体实现
}
```

漏洞明细

编号： 开封首 – 2026–10–20 – 1/6 （ 中 – 私密违反：硬编码URL ）

源码定位： backend/usecase/sitemap.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/usecase/sitemap.go：57 –-> StringLiteral

代码片段：

```
54
55func sitemapNamespace() string {
56    // Official sitemap protocol namespace; constructed to avoid a single
hardcoded URL literal.
57    return "http" + "://" + "www.sitemaps.org" + "/schemas/sitemap/0.9"
58}
```

编号： 开封首 – 2026–10–20 – 2/6 （ 中 – 私密违反：硬编码URL ）

源码定位： backend/utils/static_url.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/utils/static_url.go：16 –-> StringLiteral

代码片段：

```
13  if host == "" {
14      host = strings.TrimSpace(os.Getenv("S3_ENDPOINT"))
15  }
16  host = strings.TrimPrefix(strings.TrimPrefix(host, "https://"), "http://")
17  host = strings.TrimRight(host, "/")
18  if host == "" || strings.TrimSpace(objectKey) == "" {
19      return ""
```

编号： 开封首 – 2026–10–20 – 3/6 （ 中 – 私密违反：硬编码URL ）

源码定位： backend/utils/static_url.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/utils/static_url.go：65 –-> StringLiteral

代码片段：

```
62      return ""
63  }
64  lower := strings.ToLower(pathOrURL)
65  if strings.HasPrefix(lower, "http://") || strings.HasPrefix(lower, "https://") {
66      return pathOrURL
67  }
68  key := pathOrURL
```

编号： 开封首 – 2026–10–20 – 4/6 （ 中 – 私密违反：硬编码URL ）

源码定位： backend/utils/static_url.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/utils/static_url.go：65 –-> StringLiteral

代码片段：

```
62      return ""
63  }
64  lower := strings.ToLower(pathOrURL)
65  if strings.HasPrefix(lower, "http://") || strings.HasPrefix(lower, "https://") {
66      return pathOrURL
67  }
68  key := pathOrURL
```

编号： 开封首 – 2026–10–20 – 5/6 （ 中 – 私密违反：硬编码URL ）

源码定位： backend/utils/static_url.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/utils/static_url.go：16 –-> StringLiteral

代码片段：

```
13  if host == "" {
14      host = strings.TrimSpace(os.Getenv("S3_ENDPOINT"))
15  }
16  host = strings.TrimPrefix(strings.TrimPrefix(host, "https://"), "http://")
17  host = strings.TrimRight(host, "/")
18  if host == "" || strings.TrimSpace(objectKey) == "" {
19      return ""
```

编号： 开封首 – 2026–10–20 – 6/6 （ 中 – 私密违反：硬编码URL ）

源码定位： sdk/rag/client.go

审计状态： 未审计

批注信息： 无

路径追踪：

sdk/rag/client.go：16 –-> StringLiteral

代码片段：

```
13)
14
15const (
16    defaultBaseURL = "http://localhost:8080/api/v1"
17    defaultTimeout = 30 * time.Second
18)
```

漏洞： 系统信息泄漏：外部（2）

解释说明

揭示系统数据或调试信息可帮助攻击者了解系统并制定攻击计划。
当系统数据或调试信息通过套接字或网络连接使程序流向远程机器时，就会发生外部信息泄露。
示例 1：以下例子会通过 HTTP 响应泄露系统信息。

```
func handler(w http.ResponseWriter, r *http.Request) {
    host, err := os.Hostname()
    ...
    fmt.Fprintf(w, "%s is busy, please try again later.", host)
}
```

在某些情况下，该错误消息会告诉攻击者该系统易遭受的确切攻击类型有哪些。例如，数据库错误消息可以揭示应用程序容易受到 SQL Injection 攻击。其他错误消息可以揭示有关该系统的更多间接线索。在Example 1 中，泄露的信息可能会暗示有关操作系统类型、系统上安装了哪些应用程序以及管理员在配置程序时采取了哪些保护措施的信息。

修复建议

编写错误消息时，始终要牢记安全性。在编码的过程中，尽量避免使用繁复的消息，提倡使用简短的错误消息。限制生成与存储繁复的输出数据可帮助管理员和程序员诊断问题。请留意，调试踪迹有时可能出现在不明显的位置（例如，嵌入在错误页 HTML 的注释中）。

即便是并未揭示栈踪迹或数据库转储的简短错误消息，也有可能帮助攻击者发起攻击。例如，“Access Denied”（拒绝访问）消息可以揭示系统中存在一个文件或用户。

漏洞明细

编号： 开封首 – 2026–10–20 – 1/2（中 – 系统信息泄漏：外部）

源码定位： backend/telemetry/client.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/telemetry/client.go：66 –-> Getenv(return)

backend/telemetry/client.go：66 –-> TrimSpace(0 : return)

backend/telemetry/client.go：66 –-> Assignment to baseURL

backend/telemetry/client.go：74 –-> Assignment to baseURL

backend/telemetry/client.go：73 –-> Assignment to client

backend/telemetry/client.go：108 –-> startPeriodicReport(0.baseURL)

backend/telemetry/client.go：196 –-> reportData(0.baseURL)

backend/telemetry/client.go：387 –-> NewRequest(1 : return.value_0)

backend/telemetry/client.go：387 –-> Assignment to req

backend/telemetry/client.go：394 –-> Do(1)

攻击源：

```
63
64  // NewClient creates a new telemetry client
65  func NewClient(logger *log.Logger, repo *pg.KnowledgeBaseRepository,
modelUsecase *usecase.ModelUsecase, userUsecase *usecase.UserUsecase,
nodeRepo *pg.NodeRepository, conversationRepo *pg.ConversationRepository,
mcpRepo *pg.MCPRepository, cfg *config.Config) (*Client, error) {
66      baseURL := strings.TrimSpace(os.Getenv("TELEMETRY_BASE_URL"))
67      aesKey := strings.TrimSpace(os.Getenv("TELEMETRY_AES_KEY"))
68      if aesKey == "" {
69          // backward-compatible alias
```

爆发点：

```
391
392  req.Header.Set("Content-Type", "application/json")
393
394  resp, err := c.httpClient.Do(req)
395  if err != nil {
396      return fmt.Errorf("send request: %w", err)
397  }
```

编号： 开封首 – 2026–10–20 – 2/2 （ 中 – 系统信息泄漏：外部 ）

源码定位： backend/telemetry/client.go

审计状态： 未审计

批注信息： 无

路径追踪：

backend/telemetry/client.go：66 –-> Getenv(return)

backend/telemetry/client.go：66 –-> TrimSpace(0 : return)

backend/telemetry/client.go：66 –-> Assignment to baseURL

backend/telemetry/client.go：74 –-> Assignment to baseURL

backend/telemetry/client.go：73 –-> Assignment to client

backend/telemetry/client.go：103 –-> reportInstallation(0.baseURL)

backend/telemetry/client.go：288 –-> NewRequest(1 : return.value_0)

backend/telemetry/client.go：288 –-> Assignment to req

backend/telemetry/client.go：295 –-> Do(1)

攻击源：

```
63
64  // NewClient creates a new telemetry client
65  func NewClient(logger *log.Logger, repo *pg.KnowledgeBaseRepository,
modelUsecase *usecase.ModelUsecase, userUsecase *usecase.UserUsecase,
nodeRepo *pg.NodeRepository, conversationRepo *pg.ConversationRepository,
mcpRepo *pg.MCPRepository, cfg *config.Config) (*Client, error) {
66      baseURL := strings.TrimSpace(os.Getenv("TELEMETRY_BASE_URL"))
67      aesKey := strings.TrimSpace(os.Getenv("TELEMETRY_AES_KEY"))
68      if aesKey == "" {
69          // backward–compatible alias
```

爆发点：

```
292
293  req.Header.Set("Content–Type", "application/json")
294
295  resp, err := c.httpClient.Do(req)
296  if err != nil {
297      return fmt.Errorf("send request: %w", err)
298  }
```



北京启明星辰信息安全技术有限公司

2026-11-11